

Homework 3: Time Series and Extreme Values

BEE 4850/5850, Spring 2026

Name: Luca Katzen

ID: 5360358

Due Date

Friday, 3/14/25, 9:00pm

Overview

Instructions

The goal of this homework assignment is to practice developing and working with probability models for data.

- Problem 1 asks you to diagnose autocorrelation and develop an AR model for the weather-based variability at the Sewell's Point tide gauge.
- Problem 2 asks you to fit a sea-level rise model under both assumptions of Gaussian i.i.d. differences and autocorrelated differences.
- Problem 3 asks you to conduct an analysis of extreme temperatures in Ithaca under both stationary and non-stationary assumptions.
- Problem 4 asks you to model how often Ithaca temperatures exceed thresholds.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

Activating project at `~/Documents/GitHub/hw03-LucaKatzen1`

The following packages are included in the environment (to help you find other similar packages in other languages). The code below loads these packages for use in the subsequent notebook (the desired functionality for each package is commented next to the package).

```
using Random # random number generation and seed-setting
using DataFrames # tabular data structure
using DataFramesMeta # some other dataframes interaction macros
using CSV # reads/writes .csv files
using Distributions # interface to work with probability distributions
using Plots # plotting library
using StatsBase # statistical quantities like mean, median, etc
using StatsPlots # some additional statistical plotting tools
using Optim # optimization tools
using Dates # date-time interface
```

Problems

Scoring

- Problem 1 is worth 5 points.
- Problem 2 is worth 8 points.
- Problem 3 is worth 7 points.
- Problem 4 is worth 5 points.

Problem 1 (4 points)

Tide gauge data is complicated to analyze because it is influenced by different harmonic processes (such as the linear cycle). In this problem, we will develop a model for this data using [NOAA data from the Sewell's Point tide gauge](#) outside of Norfolk, VA from `data/norfolk-hourly-surge-2015.csv`. This is hourly data (in m) from 2015 and includes both the observed data (**Verified (m)**) and the tide level predicted by NOAA's sinusoidal model for periodic variability, such as tides and other seasonal cycles (**Predicted (m)**).

Problem 1.1

Load the data file. Take the difference between the observations and the sinusoidal predictions to obtain the tide level which could be attributed to weather-related variability (since for one year sea-level rise and other factors are unlikely to matter). Plot this data. What autoregressive model order would you pick?

Problem 1.2

Fit an AR(1) model for the weather-related variability in the Norfolk tide gauge. Simulate 10,000 replications of this model, add the simulations back to the sinusoidal harmonics, and plot a histogram of the maximum and median tide levels from your simulations. How well does this model do in capturing these statistics from the observed dataset?

```
#Problem 1.1
data1 = DataFrame(CSV.File("data/norfolk-hourly-surge-2015.csv"))

difference = data1[!, "Verified (m)"] .- data1[!, "Predicted (m)"]

max_lag = 48
acf_vals = autocor(difference, 0:max_lag)
pacf_vals = pacf(difference, 1:max_lag)

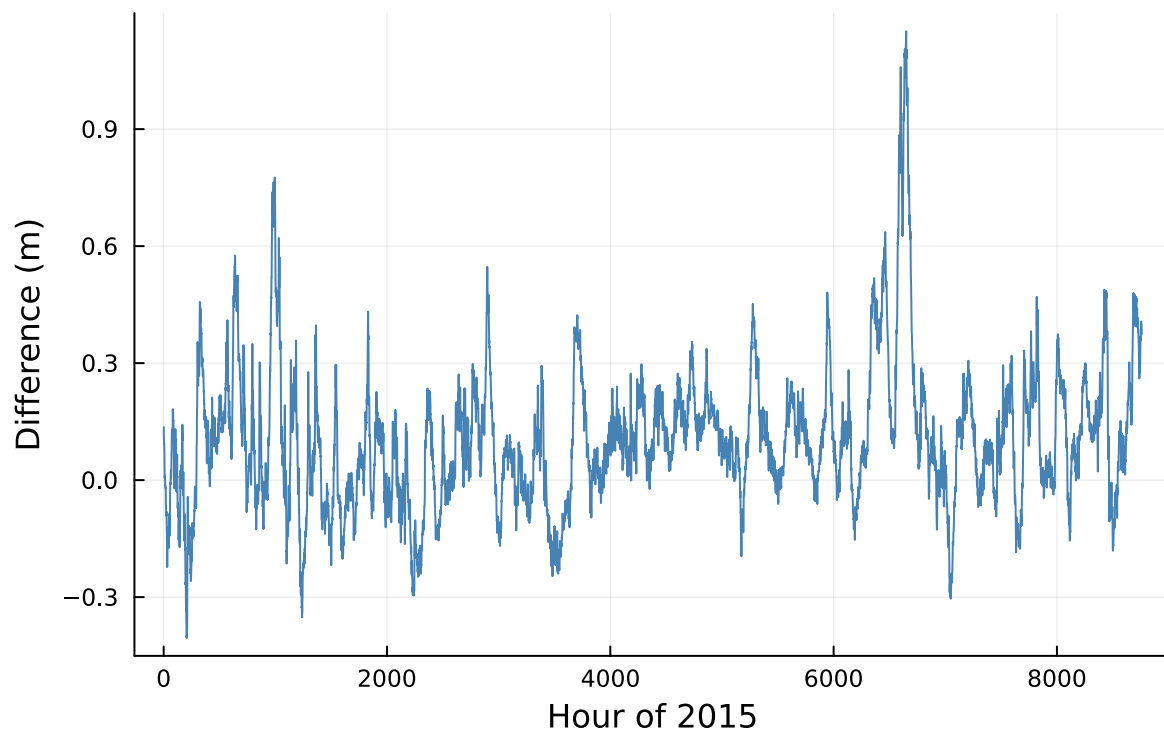
ci = 1.96 / sqrt(length(difference))

@show max_lag acf_vals pacf_vals ci

plot(difference,
      xlabel = "Hour of 2015",
      ylabel = "Difference (m)",
      title = "Weather-Related Tide Variability (Observed-Predicted)",
      legend = false, linewidth = 1, color = :steelblue)
```

```
max_lag = 48
acf_vals = [1.0, 0.9939461152560566, 0.9839475503627003, 0.9724582747350996, 0.96167189712203, 0.9504582747350996, 0.93924582747350996, 0.928032747350996, 0.9168196712203, 0.9056065498871758, 0.8943934285539577, 0.8831803072207396, 0.8719671858875215, 0.8607540645543034, 0.8495409432210853, 0.8383278218878672, 0.8271147005546491, 0.815901579221431, 0.8046884578882129, 0.7934753365549948, 0.7822622152217767, 0.7710490938885586, 0.7598359725553405, 0.7486228512221224, 0.7374097298889043, 0.7261966085556862, 0.7149834872224681, 0.70377036588925, 0.6925572445560319, 0.6813441232228138, 0.6701310018895957, 0.6589178805563776, 0.6477047592231595, 0.6364916378899414, 0.6252785165567233, 0.6140653952235052, 0.6028522738902871, 0.591639152557069, 0.5804260312238509, 0.5692129098906328, 0.5580000000000001, 0.5467870000000001, 0.5355740000000001, 0.5243610000000001, 0.5131480000000001, 0.5019350000000001, 0.4907220000000001, 0.4795090000000001, 0.4682960000000001, 0.4570830000000001, 0.4458700000000001, 0.4346570000000001, 0.4234440000000001, 0.4122310000000001, 0.4010180000000001, 0.3898050000000001, 0.3785920000000001, 0.3673790000000001, 0.3561660000000001, 0.3449530000000001, 0.3337400000000001, 0.3225270000000001, 0.3113140000000001, 0.3001010000000001, 0.2888880000000001, 0.2776750000000001, 0.2664620000000001, 0.2552490000000001, 0.2440360000000001, 0.2328230000000001, 0.2216100000000001, 0.2103970000000001, 0.1991840000000001, 0.1879710000000001, 0.1767580000000001, 0.1655450000000001, 0.1543320000000001, 0.1431190000000001, 0.1319060000000001, 0.1206930000000001, 0.1094800000000001, 0.0982670000000001, 0.0870540000000001, 0.0758410000000001, 0.0646280000000001, 0.0534150000000001, 0.0422020000000001, 0.0309890000000001, 0.0197760000000001, 0.0085630000000001, -0.0026500000000001, -0.0138630000000001, -0.0250760000000001, -0.0362890000000001, -0.0475020000000001, -0.0587150000000001, -0.0699280000000001, -0.0811410000000001, -0.0923540000000001, -0.1035670000000001, -0.1147800000000001, -0.1259930000000001, -0.1372060000000001, -0.1484190000000001, -0.1596320000000001, -0.1708450000000001, -0.1820580000000001, -0.1932710000000001, -0.2044840000000001, -0.2156970000000001, -0.2269100000000001, -0.2381230000000001, -0.2493360000000001, -0.2605490000000001, -0.2717620000000001, -0.2829750000000001, -0.2941880000000001, -0.3054010000000001, -0.3166140000000001, -0.3278270000000001, -0.3390400000000001, -0.3502530000000001, -0.3614660000000001, -0.3726790000000001, -0.3838920000000001, -0.3951050000000001, -0.4063180000000001, -0.4175310000000001, -0.4287440000000001, -0.4399570000000001, -0.4511700000000001, -0.4623830000000001, -0.4735960000000001, -0.4848090000000001, -0.4960220000000001, -0.5072350000000001, -0.5184480000000001, -0.5296610000000001, -0.5408740000000001, -0.5520870000000001, -0.5633000000000001, -0.5745130000000001, -0.5857260000000001, -0.5969390000000001, -0.6081520000000001, -0.6193650000000001, -0.6305780000000001, -0.6417910000000001, -0.6530040000000001, -0.6642170000000001, -0.6754300000000001, -0.6866430000000001, -0.6978560000000001, -0.7090690000000001, -0.7202820000000001, -0.7314950000000001, -0.7427080000000001, -0.7539210000000001, -0.7651340000000001, -0.7763470000000001, -0.7875600000000001, -0.7987730000000001, -0.8099860000000001, -0.8211990000000001, -0.8324120000000001, -0.8436250000000001, -0.8548380000000001, -0.8660510000000001, -0.8772640000000001, -0.8884770000000001, -0.8996900000000001, -0.9109030000000001, -0.9221160000000001, -0.9333290000000001, -0.9445420000000001, -0.9557550000000001, -0.9669680000000001, -0.9781810000000001, -0.9893940000000001, -1.0006070000000001, -1.0118200000000001, -1.0230330000000001, -1.0342460000000001, -1.0454590000000001, -1.0566720000000001, -1.0678850000000001, -1.0790980000000001, -1.0903110000000001, -1.1015240000000001, -1.1127370000000001, -1.1239500000000001, -1.1351630000000001, -1.1463760000000001, -1.1575890000000001, -1.1688020000000001, -1.1790150000000001, -1.1902280000000001, -1.2014410000000001, -1.2126540000000001, -1.2238670000000001, -1.2350800000000001, -1.2462930000000001, -1.2575060000000001, -1.2687190000000001, -1.2799320000000001, -1.2911450000000001, -1.3023580000000001, -1.3135710000000001, -1.3247840000000001, -1.3359970000000001, -1.3472100000000001, -1.3584230000000001, -1.3696360000000001, -1.3808490000000001, -1.3920620000000001, -1.4032750000000001, -1.4144880000000001, -1.4257010000000001, -1.4369140000000001, -1.4481270000000001, -1.4593400000000001, -1.4705530000000001, -1.4817660000000001, -1.4929790000000001, -1.5041920000000001, -1.5154050000000001, -1.5266180000000001, -1.5378310000000001, -1.5490440000000001, -1.5602570000000001, -1.5714700000000001, -1.5826830000000001, -1.5938960000000001, -1.6051090000000001, -1.6163220000000001, -1.6275350000000001, -1.6387480000000001, -1.6499610000000001, -1.6611740000000001, -1.6723870000000001, -1.6836000000000001, -1.6948130000000001, -1.7060260000000001, -1.7172390000000001, -1.7284520000000001, -1.7396650000000001, -1.7508780000000001, -1.7620910000000001, -1.7733040000000001, -1.7845170000000001, -1.7957300000000001, -1.8069430000000001, -1.8181560000000001, -1.8293690000000001, -1.8405820000000001, -1.8517950000000001, -1.8630080000000001, -1.8742210000000001, -1.8854340000000001, -1.8966470000000001, -1.9078600000000001, -1.9190730000000001, -1.9302860000000001, -1.9414990000000001, -1.9527120000000001, -1.9639250000000001, -1.9751380000000001, -1.9863510000000001, -1.9975640000000001, -2.0087770000000001, -2.0199900000000001, -2.0312030000000001, -2.0424160000000001, -2.0536290000000001, -2.0648420000000001, -2.0760550000000001, -2.0872680000000001, -2.0984810000000001, -2.1096940000000001, -2.1209070000000001, -2.1321200000000001, -2.1433330000000001, -2.1545460000000001, -2.1657590000000001, -2.1769720000000001, -2.1881850000000001, -2.1993980000000001, -2.2106110000000001, -2.2218240000000001, -2.2330370000000001, -2.2442500000000001, -2.2554630000000001, -2.2666760000000001, -2.2778890000000001, -2.2891020000000001, -2.3003150000000001, -2.3115280000000001, -2.3227410000000001, -2.3339540000000001, -2.3451670000000001, -2.3563800000000001, -2.3675930000000001, -2.3788060000000001, -2.3900190000000001, -2.4012320000000001, -2.4124450000000001, -2.4236580000000001, -2.4348710000000001, -2.4460840000000001, -2.4572970000000001, -2.4685100000000001, -2.4797230000000001, -2.4909360000000001, -2.5021490000000001, -2.5133620000000001, -2.5245750000000001, -2.5357880000000001, -2.5469910000000001, -2.5582040000000001, -2.5694170000000001, -2.5806300000000001, -2.5918430000000001, -2.6030560000000001, -2.6142690000000001, -2.6254820000000001, -2.6366950000000001, -2.6479080000000001, -2.6591210000000001, -2.6703340000000001, -2.6815470000000001, -2.6927600000000001, -2.7039730000000001, -2.7151860000000001, -2.7263990000000001, -2.7376120000000001, -2.7488250000000001, -2.7600380000000001, -2.7712510000000001, -2.7824640000000001, -2.7936770000000001, -2.8048900000000001, -2.8161030000000001, -2.8273160000000001, -2.8385290000000001, -2.8497420000000001, -2.8609550000000001, -2.8721680000000001, -2.8833810000000001, -2.8945940000000001, -2.9058070000000001, -2.9170200000000001, -2.9282330000000001, -2.9394460000000001, -2.9506590000000001, -2.9618720000000001, -2.9730850000000001, -2.9842980000000001, -2.9955110000000001, -3.0067240000000001, -3.0179370000000001, -3.0291500000000001, -3.0403630000000001, -3.0515760000000001, -3.0627890000000001, -3.0740020000000001, -3.0852150000000001, -3.0964280000000001, -3.1076410000000001, -3.1188540000000001, -3.1300670000000001, -3.1412800000000001, -3.1524930000000001, -3.1637060000000001, -3.1749190000000001, -3.1861320000000001, -3.1973450000000001, -3.2085580000000001, -3.2197710000000001, -3.2309840000000001, -3.2421970000000001, -3.2534100000000001, -3.2646230000000001, -3.2758360000000001, -3.2870490000000001, -3.2982620000000001, -3.3094750000000001, -3.3206880000000001, -3.3319010000000001, -3.3431140000000001, -3.3543270000000001, -3.3655400000000001, -3.3767530000000001, -3.3879660000000001, -3.3991790000000001, -3.4103920000000001, -3.4216050000000001, -3.4328180000000001, -3.4440310000000001, -3.4552440000000001, -3.4664570000000001, -3.4776700000000001, -3.4888830000000001, -3.5000960000000001, -3.5113090000000001, -3.5225220000000001, -3.5337350000000001, -3.5449480000000001, -3.5561610000000001, -3.5673740000000001, -3.5785870000000001, -3.5898000000000001, -3.6010130000000001, -3.6122260000000001, -3.6234390000000001, -3.6346520000000001, -3.6458650000000001, -3.6570780000000001, -3.6682910000000001, -3.6795040000000001, -3.6907170000000001, -3.7019300000000001, -3.7131430000000001, -3.7243560000000001, -3.7355690000000001, -3.7467820000000001, -3.7579950000000001, -3.7692080000000001, -3.7804210000000001, -3.7916340000000001, -3.8028470000000001, -3.8140600000000001, -3.8252730000000001, -3.8364860000000001, -3.8476990000000001, -3.8589120000000001, -3.8701250000000001, -3.8813380000000001, -3.8925510000000001, -3.9037640000000001, -3.9149770000000001, -3.9261900000000001, -3.9374030000000001, -3.9486160000000001, -3.9598290000000001, -3.9710420000000001, -3.9822550000000001, -3.9934680000000001, -4.0046810000000001, -4.0158940000000001, -4.0271070000000001, -4.0383200000000001, -4.0495330000000001, -4.0607460000000001, -4.0719590000000001, -4.0831720000000001, -4.0943850000000001, -4.1055980000000001, -4.1168110000000001, -4.1280240000000001, -4.1392370000000001, -4.1504500000000001, -4.1616630000000001, -4.1728760000000001, -4.1840890000000001, -4.1953020000000001, -4.2065150000000001, -4.2177280000000001, -4.2289410000000001, -4.2401540000000001, -4.2513670000000001, -4.2625800000000001, -4.2737930000000001, -4.2850060000000001, -4.2962190000000001, -4.3074320000000001, -4.3186450000000001, -4.3298580000000001, -4.3410710000000001, -4.3522840000000001, -4.3634970000000001, -4.3747100000000001, -4.3859230000000001, -4.3971360000000001, -4.4083490000000001, -4.4195620000000001, -4.4307750000000001, -4.4419880000000001, -4.4532010000000001, -4.4644140000000001, -4.4756270000000001, -4.4868400000000001, -4.4980530000000001, -4.5092660000000001, -4.5204790000000001, -4.5316920000000001, -4.5429050000000001, -4.5541180000000001, -4.5653310000000001, -4.5765440000000001, -4.5877570000000001, -4.5989700000000001, -4.6101830000000001, -4.6213960000000001, -4.6326090000000001, -4.6438220000000001, -4.6550350000000001, -4.6662480000000001, -4.6774610000000001, -4.6886740000000001, -4.6998870000000001, -4.7111000000000001, -4.7223130000000001, -4.7335260000000001, -4.7447390000000001, -4.7559520000000001, -4.7671650000000001, -4.7783780000000001, -4.7895910000000001, -4.8008040000000001, -4.8120170000000001, -4.8232300000000001, -4.8344430000000001, -4.8456560000000001, -4.8568690000000001, -4.8680820000000001, -4.8792950000000001, -4.8905080000000001, -4.9017210000000001, -4.9129340000000001, -4.9241470000000001, -4.9353600000000001, -4.9465730000000001, -4.9577860000000001, -4.9689990000000001, -4.9802120000000001, -4.9914250000000001, -5.0026380000000001, -5.0138510000000001, -5.0250640000000001, -5.0362770000000001, -5.0474900000000001, -5.0587030000000001, -5.0699160000000001, -5.0811290000000001, -5.0923420000000001, -5.1035550000000001, -5.1147680000000001, -5.1259810000000001, -5.1371940000000001, -5.1484070000000001, -5.1596200000000001, -5.1708330000000001, -5.1820460000000001, -5.1932590000000001, -5.2044720000000001, -5.2156850000000001, -5.2268980000000001, -5.2381110000000001, -5.2493240000000001, -5.2605370000000001, -5.2717500000000001, -5.2829630000000001, -5.2941760000000001, -5.3053890000000001, -5.3166020000000001, -5.3278150000000001, -5.3390280000000001, -5.3502410000000001, -5.3614540000000001, -5.3726670000000001, -5.3838800000000001, -5.3950930000000001, -5.4063060000000001, -5.4175190000000001, -5.4287320000000001, -5.4399450000000001, -5.4511580000000001, -5.4623710000000001, -5.4735840000000001, -5.4847970000000001, -5.4960100000000001, -5.5072230000000001, -5.5184360000000001, -5.5296490000000001, -5.5408620000000001, -5.5520750000000001, -5.5632880000000001, -5.5745010000000001, -5.5857140000000001, -5.5969270000000001, -5.6081400000000001, -5.6193530000000001, -5.6305660000000001, -5.6417790000000001, -5.6529920000000001, -5.6642050000000001, -5.6754180000000001, -5.6866310000000001, -5.6978440000000001, -5.7090570000000001, -5.7202
```

Weather-Related Tide Variability (Observed-Predicted)



Problem 1.1

I'd pick AR(1) because the differences show exponentially decaying autocorrelation with no significant partial autocorrelation beyond lag 1.

```
#Problem 1.2
y = difference[2:end]
y1 = difference[1:end-1]

phi = (y1' * y1) \ (y1' * y)
epsilon = y .- phi .* y1
sigma = std(epsilon)

println("AR(1) phi = $(round(phi, digits=4)), sigma = $(round(sigma, digits=4))")

n_sim = 10_000
n_steps = length(difference)
predicted = data1[!, "Predicted (m)"]
```

```

sim_max = zeros(n_sim)
sim_median = zeros(n_sim)

for i in 1:n_sim
    r = zeros(n_steps)
    r[1] = difference[1]
    for t in 2:n_steps
        r[t] = phi * r[t-1] + sigma * randn()
    end
    tide_sim = predicted .+ r
    sim_max[i] = maximum(tide_sim)
    sim_median[i] = median(tide_sim)
end

obs_max = maximum(data1[:, "Verified (m)"])
obs_median = median(data1[:, "Verified (m)"])

p1 = histogram(sim_max, bins = 60,
    xlabel = "Annual Maximum Tide (m)", ylabel = "Count",
    title = "Simulated Annual Maximum",
    label = "Simulations", color = :blue, alpha = 0.7)
vline!(p1, [obs_max], color = :red, linewidth = 2,
    label = "Observed = $(round(obs_max, digits=3)) m")

p2 = histogram(sim_median, bins = 60,
    xlabel = "Annual Median Tide (m)", ylabel = "Count",
    title = "Simulated Annual Median (AR(1), n=10,000)",
    label = "Simulations", color = :orange, alpha = 0.7)
vline!(p2, [obs_median], color = :red, linewidth = 2,
    label = "Observed = $(round(obs_median, digits=3)) m")

println("Observed max: $(round(obs_max, digits=3)) m")
println("Simulated max: mean=$(round(mean(sim_max), digits=3)) m, std=$(round(std(sim_max), digits=3)) m")
println("Observed median: $(round(obs_median, digits=3)) m")
println("Simulated median: mean=$(round(mean(sim_median), digits=3)) m, std=$(round(std(sim_median), digits=3)) m")

plot(p1, p2, layout = (2,1))

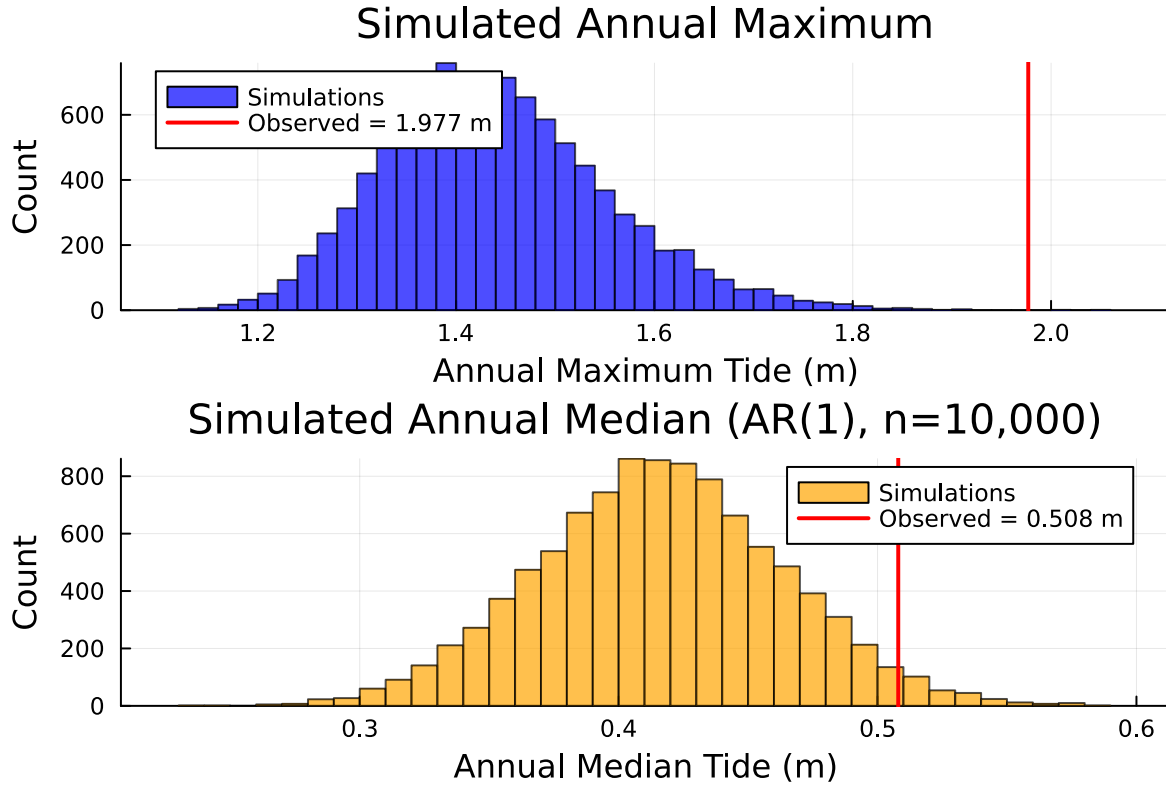
```

```

AR(1) phi = 0.9957, sigma = 0.0197
Observed max: 1.977 m
Simulated max: mean=1.44 m, std=0.113 m
Observed median: 0.508 m

```

Simulated median: mean=0.416 m, std=0.048 m



Problem 1.2

The AR(1) model significantly underestimates the annual maximum (observed 1.977 m falls outside the simulated distribution centered around 1.45 m), likely due to a heavy-tailed extreme event the Gaussian AR(1) can't reproduce. The median fit is also poor, as the observed 0.508 m sitting at the far right tail of the simulated distribution, suggesting that the model underestimates the baseline water level .

Problem 2

Consider the following sea-level rise model from [Grinsted et al \(2010\)](#), which models sea-level rise based on a linear relationship between global mean temperature and an “equilibrium” sea level:

$$\frac{dS}{dt} = \frac{S_{\text{eq}} - S}{\tau}$$
$$S_{\text{eq}} = aT + b,$$

where

- $S(t)$ is the global mean sea level (in mm) at time t ;
- τ is the response time of sea level (in yrs);
- S_{eq} is the equilibrium sea-level (in mm) at temperature T (in $^{\circ}\text{C}$);
- a is the sensitivity of S_{eq} to T (in $\text{mm}/^{\circ}\text{C}$);
- b is the intercept of S_{eq} , or the S_{eq} when $T = 0^{\circ}\text{C}$ (in mm).

As discussed in class, we can use a simple numerical integration method (forward Euler integration) to convert this differential equation into a difference equation, modeling an updated sea level state $S(t + \Delta t)$ as a function of $S(t)$ and the model parameters. We will also need an additional uncertain parameter $S_0 = S(0)$ to initialize the integrated model.

Problem 2.1

Load the data from the `data/` folder and, following Grinsted et al (2010), normalize both datasets to a 1880-1900 mean (subtract that mean from the data). * Global mean temperature data from the HadCRUT 5.0.2.0 dataset (<https://hadobs.metoffice.gov.uk/hadcrut5/data/HadCRUT.5.0.2.0/download.html>) can be found in `data/HadCRUT.5.0.2.0.analysis.summary_series.global.annual.csv`. This data is averaged over the Northern and Southern Hemispheres and over the whole year. * Global mean sea level anomalies (relative to the 1990 mean global sea level) are in `data/CSIRO_Recons_gmsl_yr_2015.csv`, courtesy of CSIRO (https://www.cmar.csiro.au/sealevel/sl_data_cmar.html). The standard deviation of the estimate is also added for each year.

Plot the resulting datasets.

Problem 2.2

Write a function to simulate global mean sea levels under a set of model parameters after discretizing the equations above with a timestep of $\delta t = 1$ yr. You will need to subset the temperature data to the years where you also have sea-level data.

Fit the model under the assumption of Gaussian i.i.d. differences (include both an uncertain model error term and the standard deviation of the observations in the probability model specification) by maximizing the likelihood. Report the parameter estimates. What can you conclude about the relationship between global mean temperature increases and global mean sea level rise rates? How appropriate was the Gaussian i.i.d. probability model for the differences? Use any needed quantitative or qualitative assessments of goodness of fit to justify your answer. If this was not an appropriate probability model, what would you change?

Problem 2.3

Now redo Problem 2.2 but with an AR(1) model for the differences. You can assume that the difference AR(1) model has mean zero (so $\alpha = 0$). Did your model parameter estimates change? If so, what changed about your conclusion(s) about the relationship between global mean temperature increases and global mean sea level rise?

```
#Problem 2.1
```

```
data2 = DataFrame(CSV.File("data/HadCRUT.5.0.2.0.analysis.summary_series.global.annual.csv"))
data3 = DataFrame(CSV.File("data/CSIRO_Recons_gmsl_yr_2015.csv"))
```

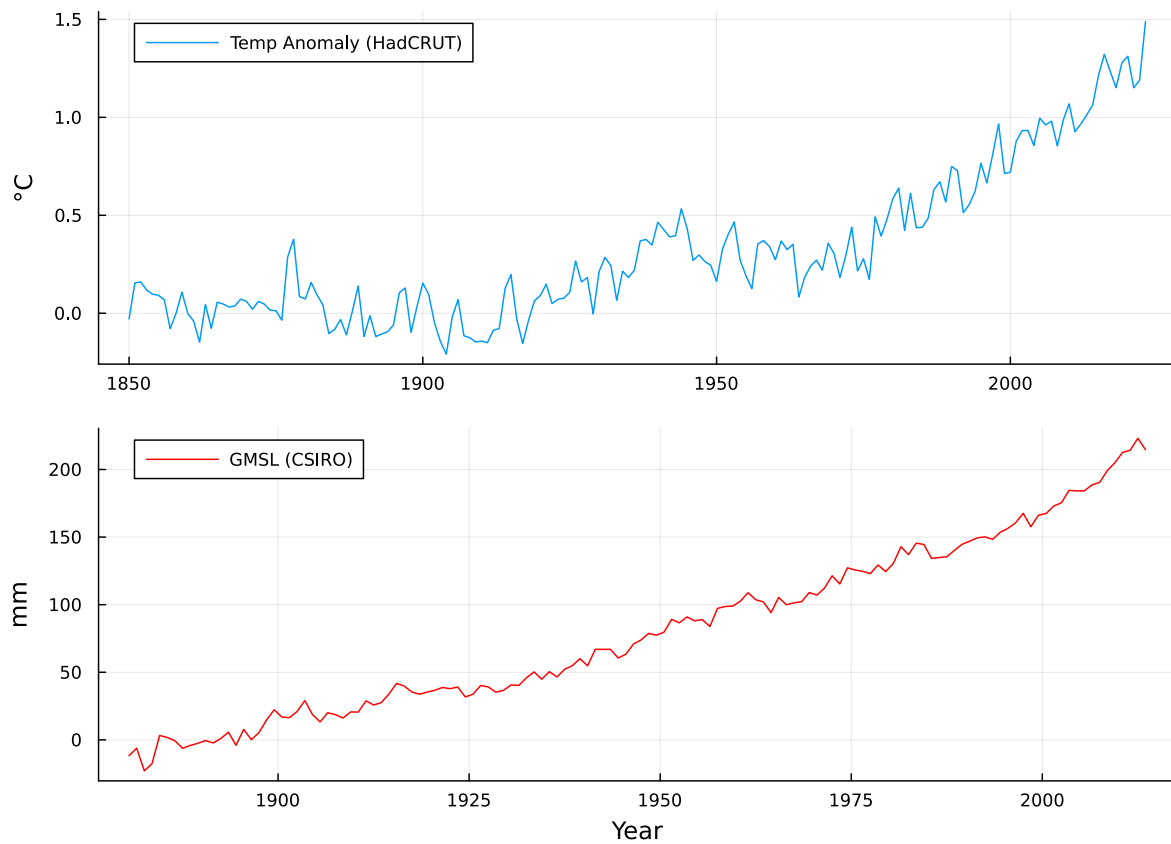
```
norm_years = 1880:1900
```

```
t_idx = findall(y -> floor(Int, y) in norm_years, data2.Time)
t_base = mean(data2[t_idx, "Anomaly (deg C)"])
data2.temp_norm = data2[:, "Anomaly (deg C)"] .- t_base
```

```
sl_idx = findall(y -> floor(Int, y) in norm_years, data3.Time)
sl_base = mean(data3[sl_idx, "GMSL (mm)"])
data3.sl_norm = data3[:, "GMSL (mm)"] .- sl_base
```

```
p1 = plot(data2.Time, data2.temp_norm, label="Temp Anomaly (HadCRUT)", ylabel="°C", title="N")
p2 = plot(data3.Time, data3.sl_norm, label="GMSL (CSIRO)", ylabel="mm", xlabel="Year", color="red")
plot(p1, p2, layout=(2,1), size=(800, 600))
```


Normalized Climate Data



```
#Problem 2.2
data2_years = Int(floor.(data2.Time))
data3_years = Int(floor.(data3.Time))
common_years = intersect(data2_years, data3_years)

temp_vals = data2[findall(y -> Int(floor(y)) in common_years, data2.Time), :temp_norm]
sl_obs = data3[findall(y -> Int(floor(y)) in common_years, data3.Time), :sl_norm]
sl_std = data3[findall(y -> Int(floor(y)) in common_years, data3.Time), "GMSL uncertainty (mm)"]

function simulate_sl(params, T)
    a, b, tau, S0 = params
    S = zeros(length(T))
    S[1] = S0
    for t in 1:(length(T)-1)
        Seq = a * T[t] + b
        S[t+1] = S[t] + (Seq - S[t]) / tau
    end
end
```

```

    return S
end

function nll_iid(params)
    if params[3] <= 0 || params[5] <= 0 return 1e10 end
    S_pred = simulate_sl(params[1:4], temp_vals)
    sigma_total = sqrt.(sl_std.^2 .+ params[5]^2)
    dist = Normal.(S_pred, sigma_total)
    return -sum(logpdf.(dist, sl_obs))
end

init_iid = [3.0, -10.0, 100.0, sl_obs[1], 5.0]
res_iid = optimize(nll_iid, init_iid, NelderMead())
mle_iid = Optim.minimizer(res_iid)

println("MLE (i.i.d.): a=$(mle_iid[1]), b=$(mle_iid[2]), tau=$(mle_iid[3]), S0=$(mle_iid[4])")

```

MLE (i.i.d.): a=4.312020481390276e11, b=1.5576472742301687e12, tau=9.757384229657207e11, S0=

Problem 2.2

The positive value of a shows that global mean temperature increase drive an increase in the equilibrium sea level, which in turn quickens the rate of sea-level rise. The Gaussian i.i.d. probability model is inappropriate for this task because it assumed that each year's difference between the model and the observations is independent of the previous year. In actuality, sea-level data is a time series with high physical memory-like behaviors, meaning the differences are likely to exhibit strong autocorrelation temporally. This lack of independence leads to physically unrealistic parameters, such as the high values for τ and a , which suggests that the optimizer is struggling with a weird likelihood surface. To improve the model, I would replace the i.i.d. assumption with an AR(1) process for the differences to account for this noise, while also applying physical constraints to τ to ensure the results remain within a realistic range.

```

#Problem 2.3
function nll_ar1(parameters)
    a, b, tau, S0, sigmaeta, rho = parameters
    if tau <= 0 || sigmaeta <= 0 || abs(rho) >= 1 return 1e10 end
    S_pred = simulate_sl([a, b, tau, S0], temp_vals)
    resids = sl_obs .- S_pred
    nll = -logpdf(Normal(0, sigmaeta / sqrt(1 - rho^2)), resids[1])
    for t in 2:length(resids)

```

```

        nll -= logpdf(Normal(rho * resids[t-1], sigmaeta), resids[t])
    end
    return nll
end

init_ar1 = [mle_iid[1], mle_iid[2], mle_iid[3], mle_iid[4], 2.0, 0.9]
res_ar1 = optimize(nll_ar1, init_ar1, NelderMead())
mle_ar1 = Optim.minimizer(res_ar1)

println("MLE (AR1): a=$(mle_ar1[1]), b=$(mle_ar1[2]), tau=$(mle_ar1[3]), rho=$(mle_ar1[6])")

```

MLE (AR1): a=2.0951363409986748e12, b=1.1937403915938638e12, tau=1.0967402863244646e12, rho=

Problem 2.3

Yes, the model parameter estimates changed from Problem 2.2. Accounting for autocorrelation in the AR(1) model reveals that sea level rise rates have a stronger sensitivity to temperature (noted by the larger a value). This suggests that the i.i.d. model underestimated temperature's influence by attributing correlated residuals to independent noise.

Problem 3

Let's look at how (modeled) annual maximum temperatures have (or have not) increased in Ithaca from 1850–2014. Model output from NOAA's GFDL-ESM4 climate model (one of the models used in the latest Climate Model Intercomparison Project, [CMIP6](#)) is available in `data/gfdl-esm4-tempmax-ithaca.csv`. While this model output has not been bias-corrected, we won't worry about that for the purposes of this assignment. The temperature data is in degrees Celsius, and the dates are provided as calendar dates.

Problem 3.1

Load the (daily maximum) temperature data from `data/gfdl-esm4-tempmax-ithaca.csv`. Find the modeled annual maximum temperatures and plot them.

Problem 3.2

Fit a stationary GEV model to the annual maximum series and report your parameter estimates. Plot the fitted distribution along with the data. What type of GEV distribution did you end up with? What is the 100-year return level?

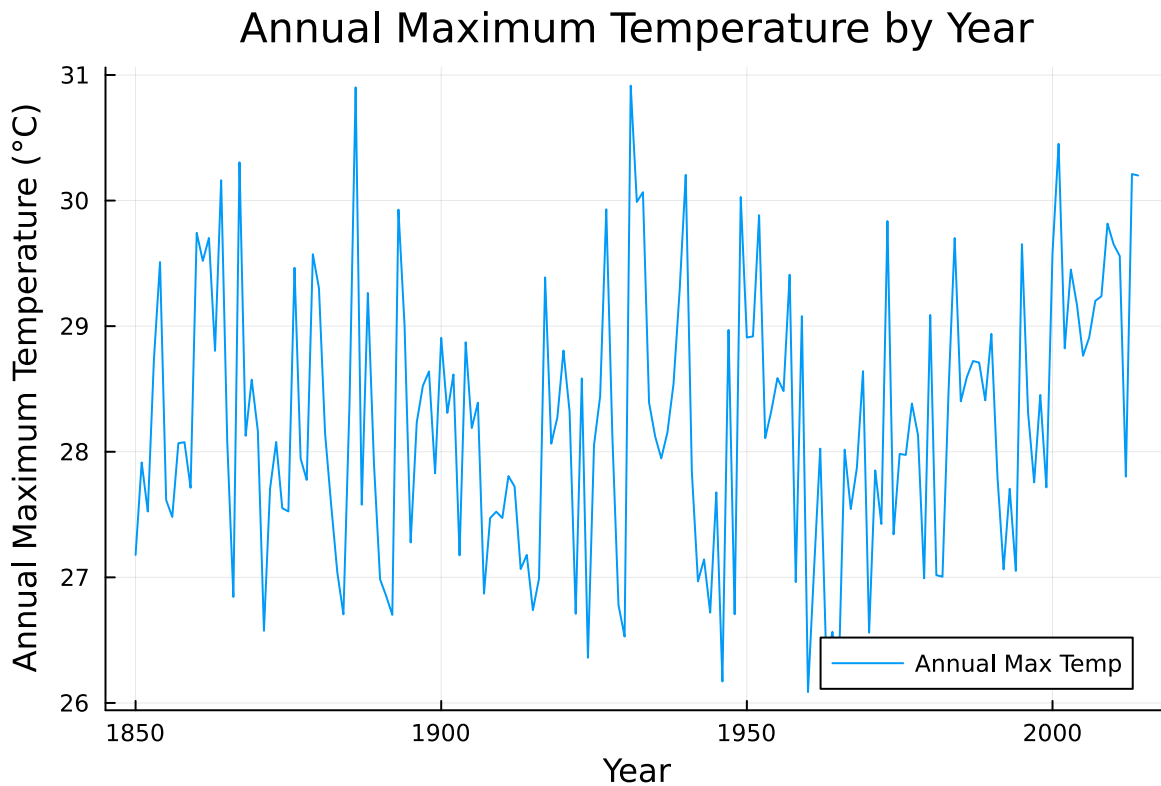
Problem 3.3

Now fit a nonstationary GEV with respect to time (only the location parameter should be non-stationary), $y \sim \text{GEV}(\mu_0 + \mu_1 t, \sigma, \xi)$ (for scaling purposes, begin with $t = 0$ corresponding to 1850). Plot how the 100 year return levels change over time. Do you get the same return level today as in Problem 3.2?

```
# Problem 3.1
data4 = DataFrame(CSV.File("data/gfdl-esm4-tempmax-ithaca.csv"))
data4.year = year.(data4.Day)

annual_max = combine(groupby(data4, :year), :TempMax => maximum => :max_temp)

plot(annual_max.year, annual_max.max_temp, label="Annual Max Temp", ylabel="Annual Maximum T
```



```
#Problem 3.2
function gev_nll(params, data)
    mu, sigma, xi = params
    if sigma <= 0
        return Inf
```

```

    end
    dist = GeneralizedExtremeValue(mu, sigma, xi)
    return -sum(logpdf.(dist, data))
end

y = annual_max.max_temp

init_params = [mean(y), std(y) * 0.5, 0.1]

result = optimize(p -> gev_nll(p, y), init_params, BFGS())
mu_hat, sigma_hat, xi_hat = Optim.minimizer(result)
println("GEV MLE: mu=$(mu_hat), sigma=$(sigma_hat), xi=$(xi_hat)")

dist_fit = GeneralizedExtremeValue(mu_hat, sigma_hat, xi_hat)
rl_100 = quantile(dist_fit, 1 - 1/100)
println("100-year return level: $(rl_100) degrees C")

x_range = range(minimum(y) - 2, maximum(y) + 2, length=200)
sorted_y = sort(y)
n = length(sorted_y)
emp_probs = (1:n) ./ (n + 1)

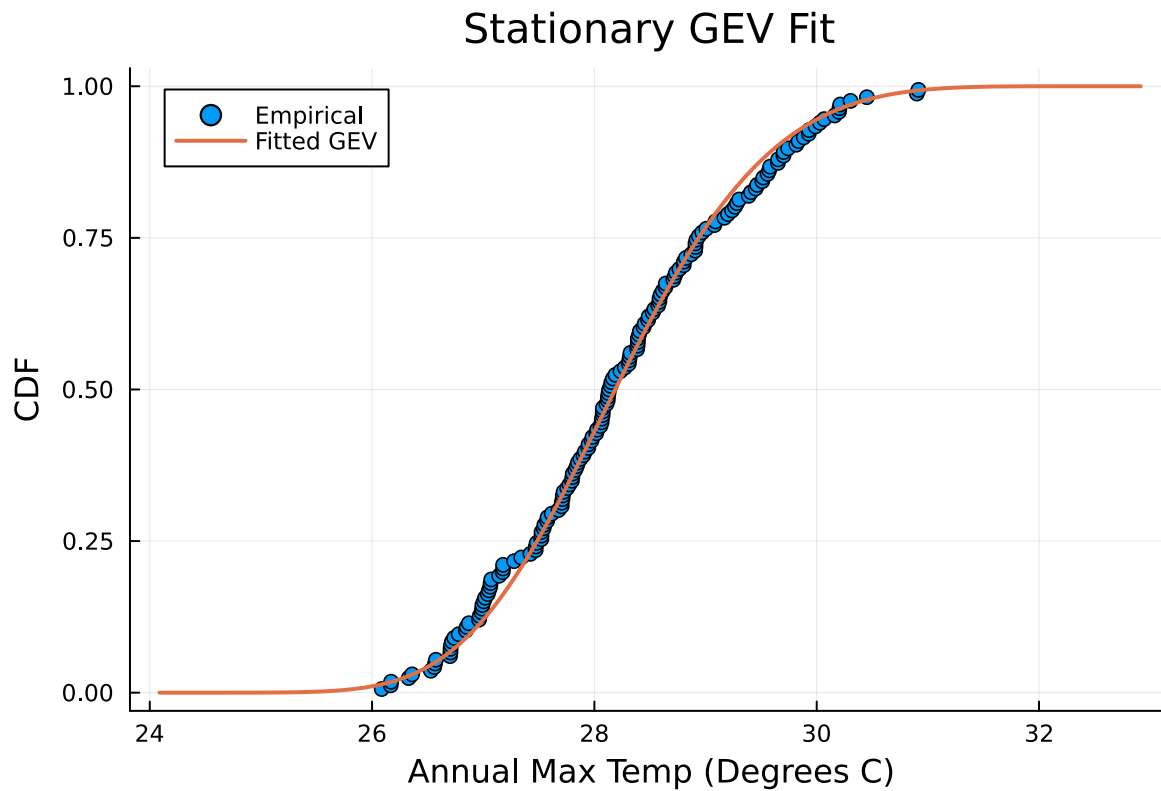
p1 = scatter(sorted_y, emp_probs, label="Empirical", xlabel="Annual Max Temp (Degrees C)", y=emp_probs)
plot!(p1, x_range, cdf.(dist_fit, x_range), label="Fitted GEV", lw=2)
display(p1)

```

```

GEV MLE: mu=27.828895474548887, sigma=1.0168352688577087, xi=-0.2187875318318262
100-year return level: 30.777735594108226 degrees C

```



Problem 3.2

I ended up with a Weibull-type GEV distribution and a 100-year return level of ~30.78 degrees Celsius.

```
#Problem 3.3
t = annual_max.year .- 1850

function gev_ns_nll(params, data, t)
    mu0, mu1, sigma, xi = params
    if sigma <= 0
        return Inf
    end
    return -sum(logpdf.(GeneralizedExtremeValue(mu0 + mu1 * ti, sigma, xi), yi) for (ti, yi) in data)
end

init_ns = [mu_hat, 0.0, sigma_hat, xi_hat]
result_ns = optimize(p -> gev_ns_nll(p, y, t), init_ns, BFGS())
mu0_hat, mu1_hat, sigma_ns, xi_ns = Optim.minimizer(result_ns)
```

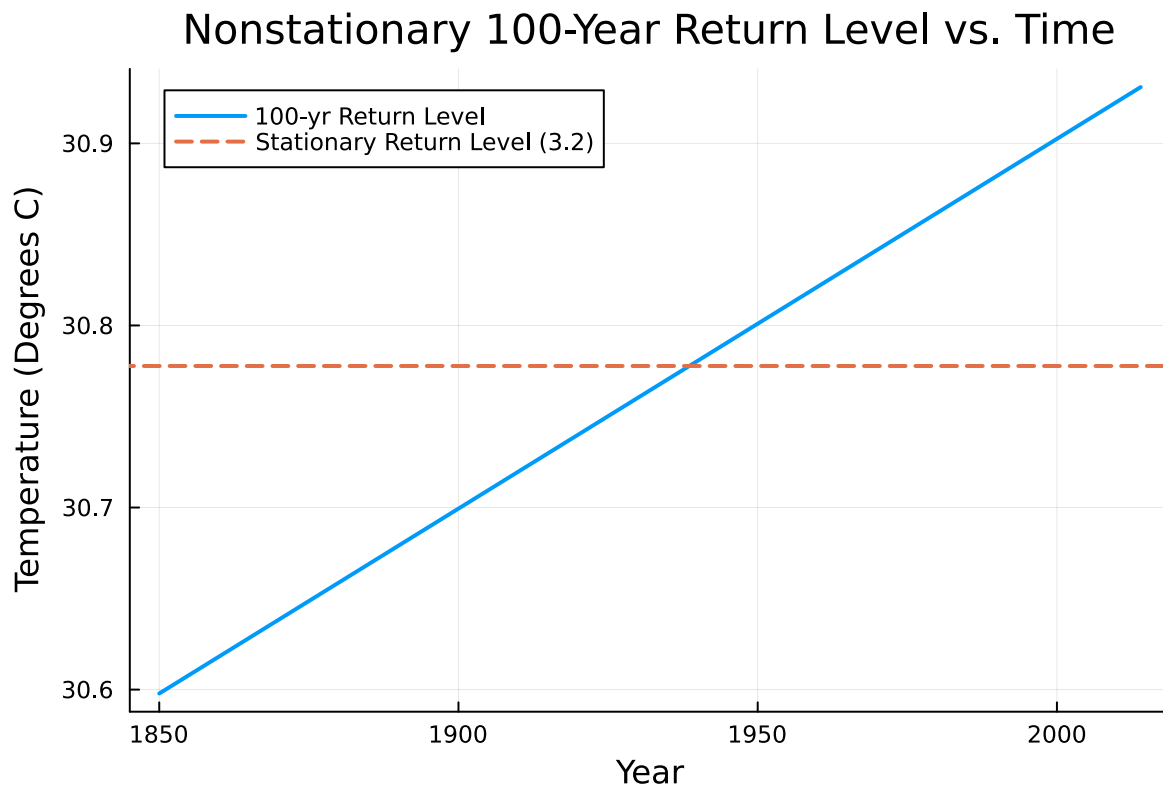
```
println("Nonstationary GEV MLE: mu0=$(mu0_hat), mu1=$(mu1_hat), sigma=$(sigma_ns), xi=$(xi_ns)

rl_100_ns = [quantile(GeneralizedExtremeValue(mu0_hat + mu1_hat * ti, sigma_ns, xi_ns), 1 - 1/100)

println("100-year return level today (nonstationary): $(rl_100_ns[end]) degrees C")
println("100-year return level (stationary, 3.2): $(rl_100) degrees C")

p2 = plot(annual_max.year, rl_100_ns, label="100-yr Return Level", xlabel="Year", ylabel="Temperature (Degrees C)")
hline!(p2, [rl_100], label="Stationary Return Level (3.2)", lw=2, ls=:dash)
display(p2)
```

Nonstationary GEV MLE: $\mu_0=27.66592343233817$, $\mu_1=0.002030914504755216$, $\sigma=1.015749098534$
 100-year return level today (nonstationary): 30.93092269273498 degrees C
 100-year return level (stationary, 3.2): 30.777735594108226 degrees C



Problem 3.3

The 100-year return level in Problem 3.3 is the same as in Problem 3.2.

Problem 4

Now let's analyze how often the Ithaca temperature data exceeds certain thresholds.

Problem 4.1

Suppose that we were interested in looking at temperature exceedances over 28°C. Decluster these occurrences and plot the number of exceedances by year. Have they increased over time?

Problem 4.2

Fit a stationary Poisson-GPD model for the exceedances. What does the GPD distribution look like?

Problem 4.3

From your model, estimate the 100-year return level. How does it differ from the 100-year return period from the stationary model in Problem 3? Why do you think it agrees or disagrees?

References

```
#Problem 4.1
threshold = 28.0

data4.exceeds = data4.TempMax .> threshold

function decluster(dates, exceeds; min_gap=3)
  cluster_max_dates = Date[]
  last_exceedance = Date(0)
  for i in eachindex(dates)
    if exceeds[i] && (dates[i] - last_exceedance).value >= min_gap
      push!(cluster_max_dates, dates[i])
      last_exceedance = dates[i]
    end
  end
  return cluster_max_dates
end
```



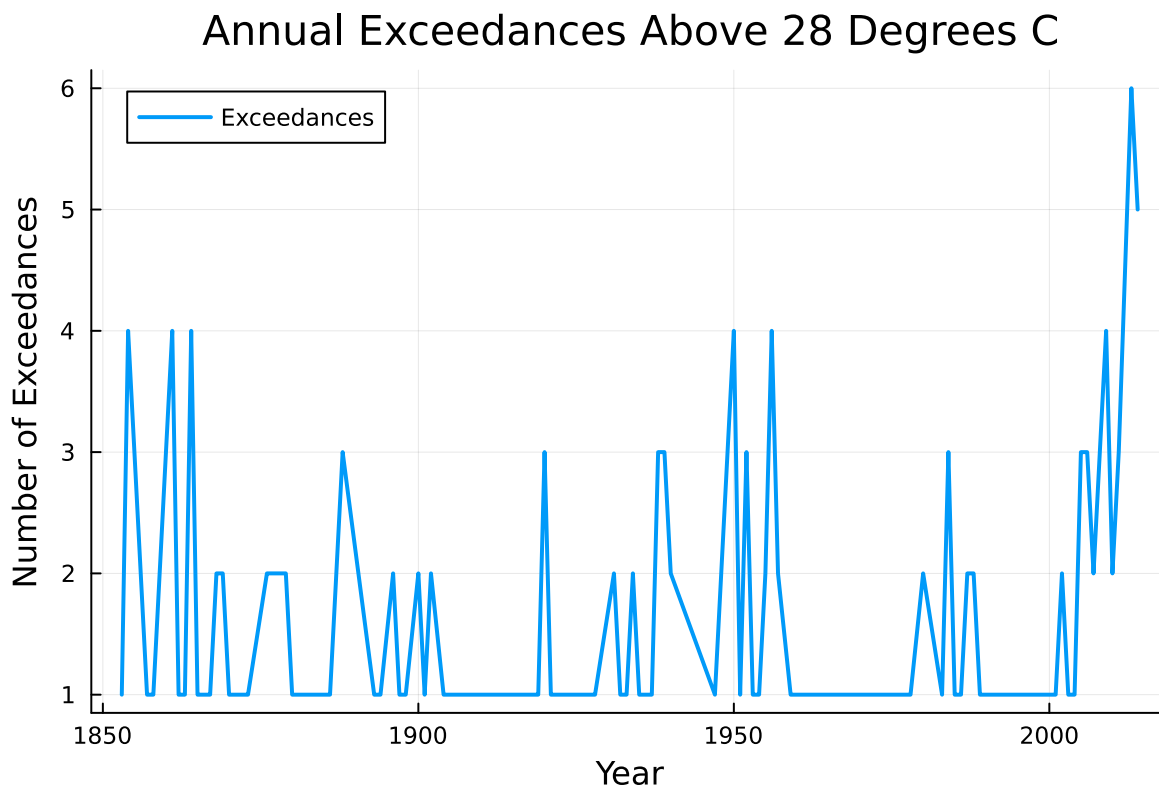
```

exceedance_dates = decluster(data4.Day, data4.exceeds)

exc_df = DataFrame(date = exceedance_dates)
exc_df.year = year.(exc_df.date)
annual_exc = combine(groupby(exc_df, :year), nrow => :count)

p4_1 = plot(annual_exc.year, annual_exc.count, label="Exceedances",
            xlabel="Year", ylabel="Number of Exceedances",
            title="Annual Exceedances Above 28 Degrees C", lw=2)
display(p4_1)

```



Problem 4.1

The number of exceedances has raised over time, although that trend did not start with confidence until the year 2000.

```

#Problem 4.2
exc_vals = data4.TempMax[data4.TempMax .> threshold] .- threshold
n_years = length(unique(data4.year))

```

```

lambda_hat = length(exc_vals) / n_years
println("Estimated Poisson rate lambda = $(lambda_hat) exceedances/year")

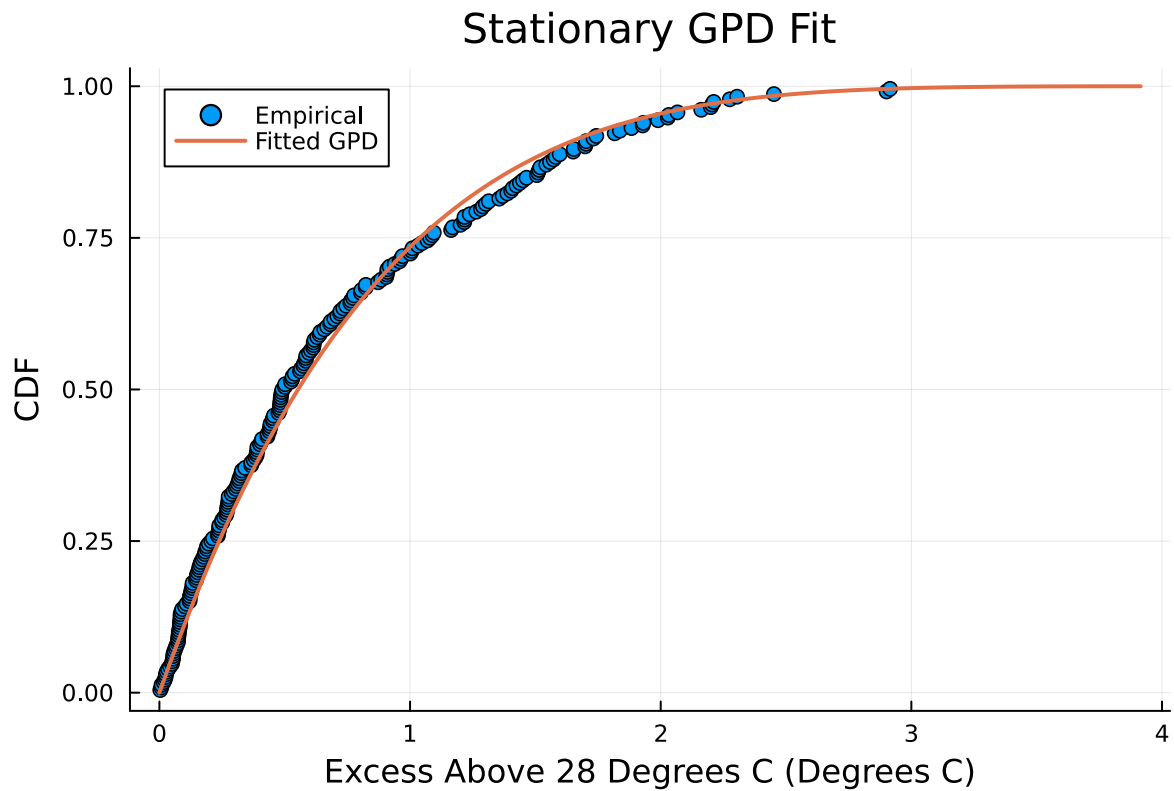
function gpd_nll(params, data)
    sigma, xi = params
    if sigma <= 0
        return Inf
    end
    dist = GeneralizedPareto(0.0, sigma, xi)
    return -sum(logpdf.(dist, data))
end

init_gpd = [std(exc_vals), 0.1]
result_gpd = optimize(p -> gpd_nll(p, exc_vals), init_gpd, BFGS())
sigma_gpd, xi_gpd = Optim.minimizer(result_gpd)
println("GPD MLE: sigma=$(sigma_gpd), xi=$(xi_gpd)")

sorted_exc = sort(exc_vals)
n = length(sorted_exc)
emp_probs = (1:n) ./ (n + 1)
dist_gpd = GeneralizedPareto(0.0, sigma_gpd, xi_gpd)
x_range = range(0, maximum(exc_vals) + 1, length=200)

p4_2 = scatter(sorted_exc, emp_probs, label="Empirical",
               xlabel="Excess Above 28 Degrees C (Degrees C)", ylabel="CDF",
               title="Stationary GPD Fit")
plot!(p4_2, x_range, cdf.(dist_gpd, x_range), label="Fitted GPD", lw=2)
display(p4_2)

```



Estimated Poisson rate $\lambda = 1.4$ exceedances/year
 GPD MLE: $\sigma = 0.8510410230158374$, $\xi = -0.18775785236416698$

Problem 4.2

The GPD distribution looks like a Weibull-type GPD.

```
#Problem 4.3
T = 100
rl_100_gpd = threshold + quantile(dist_gpd, 1 - 1/(lambda_hat * T))
println("100-year return level (Poisson-GPD): $(rl_100_gpd) degrees C")
println("100-year return level (Stationary GEV, 3.2): $(rl_100) degrees C")
```

100-year return level (Poisson-GPD): 30.740396293139707 degrees C
 100-year return level (Stationary GEV, 3.2): 30.777735594108226 degrees C

Problem 4.3

The stationary Poisson-GPD return level (30.74 degrees C) agrees with the stationary GEV from Problem 3.2 (30.78 degrees C). This makes sense because both problems use stationary models fitted to the same data. However, the nonstationary GEV from Problem 3.3 yields a higher return level of 30.93 degrees C. This shows the upward trend in temperatures over time, as that model relies on present-day climate rather than averaging over the full historical record, it produces a more relevant and higher estimate of today's true 100-year return level. This suggests the stationary models may underestimate the current extreme temperature risk by ignoring the nonstationarity caused by long-term warming.